Authentication & JWT in React

3 min read · Updated 24 Mar 2026 · Topic 38 of 62

Bookmark Mark done

8.3 – Authentication & JWT in React

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"How do you handle authentication in React?"	🟡🔴	★★★★★
"What is JWT?"	🟡	★★★★☆
"Where do you store tokens?"	🟡🔴	★★★★☆
"How do you protect API routes on the client?"	🟡	★★★☆☆

The JWT Authentication Flow

1. User submits login credentials (email + password)

2. Server validates credentials and returns a JWT (JSON Web Token)
3. Client stores the token
4. Client sends the token with every API request via the `Authorization: Bearer <token>` header
5. Server validates the token on each request and returns data or 401

Login implementation

jsx

JSX Copy

```jsx
async function login(email, password) {
  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });

  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.message || 'Login failed');
  }

  const { token, user } = await response.json();
  localStorage.setItem('token', token);
  return user;
}
```

Authenticated API calls

jsx

JSX Copy

```jsx
async function fetchWithAuth(url, options = {}) {
  const token = localStorage.getItem('token');

  const response = await fetch(url, {
```

```
      'Content-Type': 'application/json',
      Authorization: token ? `Bearer ${token}` : '',
    },
  });

  if (response.status === 401) {
    localStorage.removeItem('token');
    window.location.href = '/login';
    throw new Error('Session expired');
  }

  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}
```

AuthContext pattern

jsx

JSX Copy

```
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
      fetch('/api/auth/me', {
        headers: { Authorization: `Bearer ${token}` },
      })
        .then(r => r.ok ? r.json() : Promise.reject())
        .then(setUser)
        .catch(() => localStorage.removeItem('token'))
        .finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
```

```
  const login = async (email, password) => {
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    });
    const { token, user } = await res.json();
    localStorage.setItem('token', token);
    setUser(user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  if (loading) return <FullPageSpinner />;

  return (
    <AuthContext.Provider value={{ user, login, logout, isAuthenticated: !
      {children}
    </AuthContext.Provider>
  );
}

function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used inside AuthProvider'
  return context;
}
```

Token storage comparison

	localStorage	httpOnly cookie
JS access	Yes (vulnerable to XSS)	No (more secure)

XSS vulnerability	High	Low
CSRF vulnerability	Low	High (mitigate with CSRF tokens)
Setup complexity	Simple	Requires backend cooperation

For interviews, localStorage is the standard answer. Mention httpOnly cookies as the more secure production option.

✅ Key Takeaways

- JWT flow: login, receive token, store it, attach to every request, handle 401.
- Use **AuthContext + useAuth hook** for app-wide auth state.
- **localStorage** for simplicity. **httpOnly cookies** for production security.
- Always handle **token expiration** (401 responses) with redirect to login.
- Show a **loading spinner** while validating the token on initial load.
- Client-side auth is **UX only**. Server must validate tokens independently.

QUICK CHECK — no marks, just for you.

1. In the lesson's fetchWithAuth helper, what happens when the server responds with status 401?

- ◯ The request is silently retried once with the same token before failing
- ◯ A refresh token is exchanged for a new access token and the original request replays

- [ ] The stored token is removed, the browser is redirected to /login, and an error is thrown

Check answer

2. According to the lesson's token storage comparison, what is the main trade-off of moving the JWT from localStorage into an httpOnly cookie?

- [ ] XSS exposure drops because JavaScript cannot read it, but CSRF risk rises and must be mitigated with CSRF tokens
- [ ] Security is identical to localStorage; the only difference is that setup is simpler
- [ ] CSRF risk drops because the cookie is sent automatically, but XSS risk rises since scripts can read cookies
- [ ] Both XSS and CSRF risk drop to zero, which is why it needs no extra mitigation

Check answer

3. The lesson says client-side auth is UX only. Which statement best explains why guarding routes with isAuthenticated from useAuth is not real security?

- [ ] localStorage is cleared on refresh, so the client can never be sure the user is still logged in
- [ ] The server independently validates the token on every request and returns data or 401, so it is the real gate
- [ ] React Context state is public, so any other component can overwrite the user object
- [ ] JWTs are encrypted, so only the client that created them can read their contents

Check answer

Previous
Loading, Error & Empty States

Next
Debouncing & Throttling

LEARN MORE

ARTICLES & DOCS

- Introduction to JWT
  jwt.io
- React Router — home & docs
  reactrouter.com

VIDEOS



JWT auth: access & refresh tokens
YouTube



JWT authentication (React + Node)
YouTube



Storing a JWT in a cookie
YouTube

Hand-picked references for this lesson.